

Topic 12.4 – Mughal Architecture Identifier (Tier 1)

SMAI Assignment 3 – IIIT Hyderabad, 2025–26

Mohammed Sami – 2025121005 (mohammed.sami@research.iiit.ac.in)
Shreyash Chandak – 2025121006 (shreyash.chandak@research.iiit.ac.in)

GitHub Repository: <https://github.com/shreyash-chandak/mughlaiMalaiTikka-smaiA3>
Hugging Face Model: <https://huggingface.co/platynator/clip-mughal-model/tree/main>
Deployed on Streamlit: <https://mughalarchitectureclassifier.streamlit.app/>

Contents

1	Introduction	2
2	Data	2
2.1	Custom Dataset Collection	2
2.2	Class Imbalance	3
2.3	Offline Augmentation	3
3	Method (& The fundamental challenges in our problem)	4
3.1	The Similar-Prompt Problem	4
3.2	Confusion Clusters	5
3.3	Improvement 1: Prompt Ensembling and Multi-View Scoring	6
3.4	Improvement 2: Specialist Fine-Tuning	7
3.4.1	What We Fine-Tune (and What We Don't)	7
3.4.2	Training Objective: Why Not Contrastive Loss?	7
3.4.3	Freeze Policy	7
3.4.4	Label Smoothing	7
3.4.5	Hybrid Inference Pipeline	8
3.5	The Nested Monument Problem	9
3.6	Deployment	9
4	Results	11
4.1	Where Our Model Still Fails	13
5	Limitations and Future Work	15
6	References	15

1 Introduction

A tourist points their phone at a Mughal monument – the app should identify it, surface its history, and offer a Google Maps link. Simple enough for the Taj Mahal. But what about Bibi Ka Maqbara, a monument *deliberately built as a replica* of the Taj? Or Agra Fort versus Red Fort – two massive red sandstone fortifications separated by 200 km but sharing the same Mughal architectural DNA?

This is fundamentally a **fine-grained visual classification** problem. Unlike distinguishing a cat from a dog, our 15 classes share a common visual vocabulary: arched gateways, marble inlay, red sandstone walls, symmetrical gardens, and onion domes. The challenge is not recognizing *Mughal architecture* – it is telling apart monuments that were designed by the same dynasty, often by the same architects, using the same materials.

We build a hybrid system: zero-shot CLIP for easy classes, a fine-tuned specialist for confusable clusters, deployed as a Streamlit app with rich contextual information.

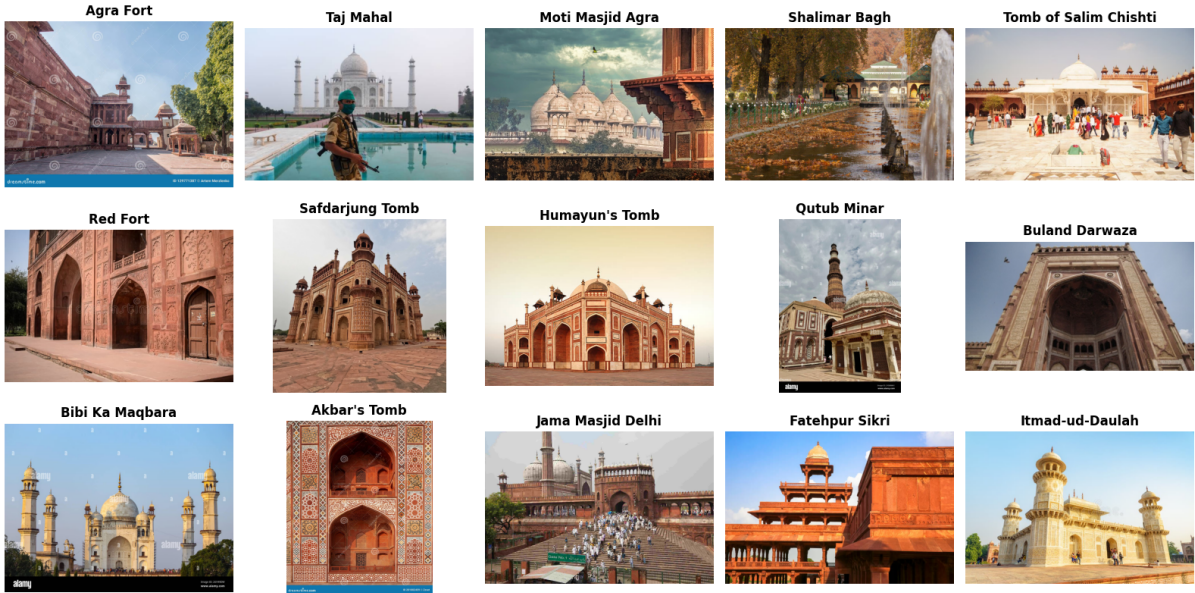


Figure 1: Representative images from our custom-curated dataset of 15 Mughal monuments. Note the visual similarity across classes – red sandstone dominates multiple categories, and white marble appears in at least five.

2 Data

2.1 Custom Dataset Collection

The dataset provided in the problem statement (Kaggle: *Indian Monuments Image Dataset*) primarily focuses on a small subset of highly popular landmarks such as **Humayun’s Tomb**, **Taj Mahal**, **Qutub Minar**, and **Fatehpur Sikri**. Moreover, several classes in that dataset are not Mughal monuments, making it unsuitable for a fine-grained, Mughal-specific classification task. Crucially, many architecturally significant Mughal structures - such as **Akbar’s Tomb**, **Itmad-ud-Daulah**, and others in our target list of 15 monuments, are either completely absent or severely underrepresented, not only in this dataset but across other publicly available sources as well.

To address this gap, we **curated our own dataset**, consisting of **796 real images** across 15 Mughal monuments. Images were manually collected and verified to ensure correct labeling and

visual clarity. While we aimed to maintain consistency by prioritizing frontal views (as they best capture defining architectural features), we also included side angles, low-angle shots, and varied perspectives to improve robustness and generalization under real-world conditions.

Why build a custom dataset?

Existing datasets are biased toward a few iconic monuments and fail to capture the diversity within Mughal architecture. For fine-grained classification, the challenge lies in distinguishing *subtle visual differences*: the lattice patterns of Itmad-ud-Daulah vs. Tomb of Salim Chishti, the rampart profiles of Red Fort vs. Agra Fort, or the dome proportions of Humayun’s Tomb vs. Safdarjung Tomb. Building a custom dataset allowed us to explicitly target these distinctions and ensure balanced representation across all classes.

2.2 Class Imbalance

Class imbalance is inherent to the problem. The Taj Mahal (187 images) is the most photographed building in India; Akbar’s Tomb (36 images) is a regional tourist site. This imbalance reflects real-world data availability, not careless collection.

2.3 Offline Augmentation

To address imbalance, we applied **offline augmentation** to bring under-represented classes up to 80 images each:

- `RandomResizedCrop(224, scale=(0.65, 1.0))`
- `RandomHorizontalFlip()`
- `RandomRotation(10)`
- `ColorJitter(brightness=0.15, contrast=0.15, saturation=0.10, hue=0.04)`
- `GaussianBlur(kernel_size=3)`

Design Decision: Augmentation Leakage Prevention

Augmented images were **only added to training folds** - never to validation or test splits. An augmented copy is visually near-identical to its source; if both appear in train and test, the model is effectively tested on images it has already seen. We tag all augmented files with an `aug_` prefix and filter them out during fold construction. Without this separation, accuracy can be inflated by 10–15%.

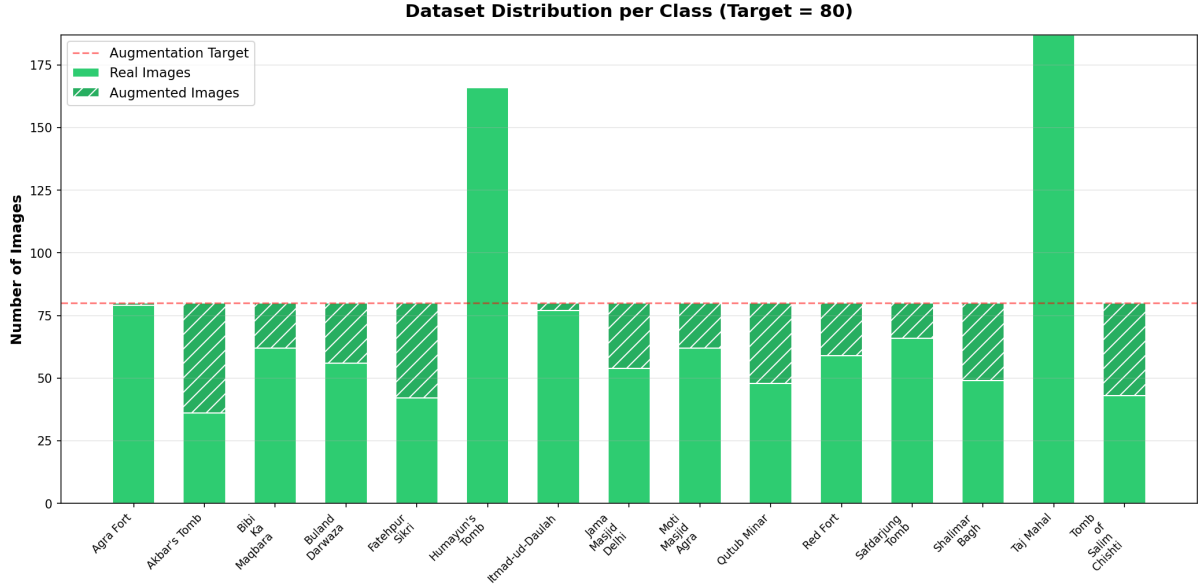


Figure 2: Image count per class. Bars show real images (light) and augmented images (dark). Classes like Akbar’s Tomb required significant augmentation to reach the 80-image target.

3 Method (& The fundamental challenges in our problem)

3.1 The Similar-Prompt Problem

CLIP classifies by comparing an image embedding against text embeddings of class prompts. For most tasks, the prompts are semantically distant (“a photo of a cat” vs. “a photo of a car”). But for Mughal architecture, **the prompts themselves become confusable**:

“A photo of the **Red Fort**, a large red sandstone Mughal fortification”
 “A photo of **Agra Fort**, a large red sandstone Mughal fortification”

These two prompts differ by only a single proper noun. CLIP’s text encoder maps them to nearly identical embeddings, making visual discrimination almost impossible from the text side alone. The model must rely entirely on subtle visual differences – and with a single prompt per class, it has very little signal to work with.

Core Insight

This is the **fundamental challenge of our problem**: unlike standard image classification where class *names* are semantically distinct, our class names describe variations of the same architectural tradition. The text modality, which is CLIP’s strength, becomes a weakness here.



Figure 3: Visually similar monument pairs. Bibi Ka Maqbara (right) was built in 1660 as an explicit imitation of the Taj Mahal (left). Even humans require contextual cues (city, surrounding structures) to distinguish them reliably.

3.2 Confusion Clusters

The base CLIP confusion matrix (66.76% top-1) reveals structured, *architecturally motivated* error patterns:

True Class	Predicted As	Count	Why
Agra Fort	Buland Darwaza	31	Both red sandstone; fort has gateway-like walls
Red Fort	Buland Darwaza	35	Red sandstone catch-all
Safdarjung ↔ Humayun's	Each other	35	Safdarjung was <i>inspired by</i> Humayun's
Itmad-ud-Daulah	Salim Chishti	13	Both small ornate white marble tombs

Table 1: Top confusion pairs from base CLIP. These are not random errors – they reflect genuine architectural similarity.

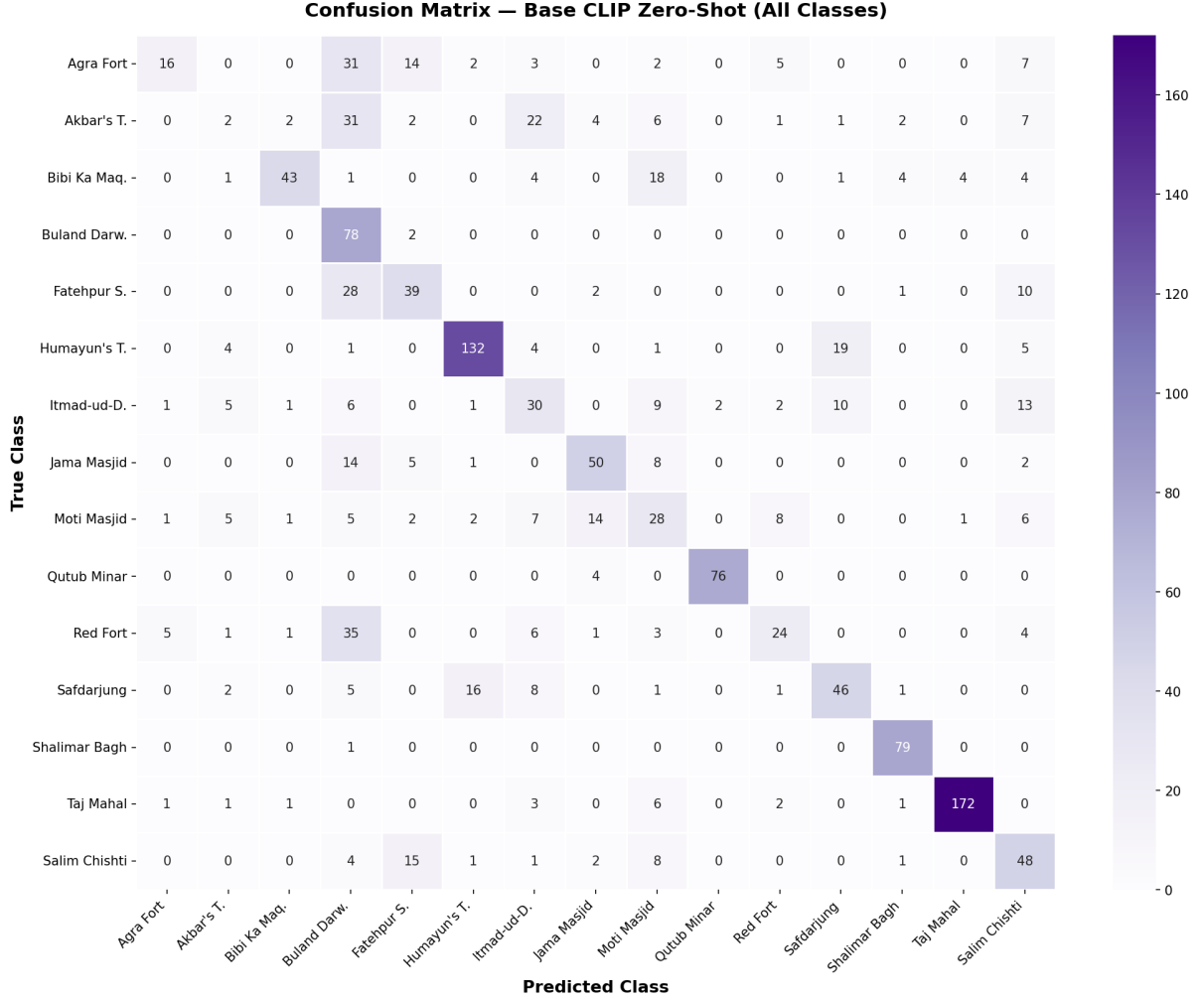


Figure 4: Confusion matrix for base CLIP zero-shot classification. Note the heavy off-diagonal mass in the red sandstone cluster (Agra Fort, Red Fort, Buland Darwaza).

3.3 Improvement 1: Prompt Ensembling and Multi-View Scoring

Our first improvement stays within the zero-shot constraint – no training required.

Prompt ensembling. Instead of one prompt per class, we craft 5–8 descriptive prompts that emphasize different architectural features:

“A photo of the Red Fort, a massive Mughal fortress with **red sandstone ramparts**”
 “A photo of the Red Fort, featuring **octagonal towers and defensive walls** in Delhi”
 “The Red Fort, a **Shahjahanabad citadel** with Lahore Gate and Naubat Khana”

By averaging the text embeddings of these prompts, we create a richer class representation that captures multiple visual aspects. Different photos of the Red Fort may show ramparts, gates, or interior palaces – a single generic prompt cannot cover all of these.

Multi-view scoring. We generate 6 augmented views of the input image (original, autocontrast, sharpened, contrast-enhanced, center crop, close crop) and average their embeddings. This makes the system robust to lighting, framing, and exposure variations.

Family-aware reranking. We add a coarse classifier that groups monuments into architectural families (tomb, fort, mosque, garden, minaret, complex). When two classes are close in score, the family signal can break the tie.

3.4 Improvement 2: Specialist Fine-Tuning

Prompt engineering hits a ceiling when the classes are genuinely visually similar. The text embeddings for “Red Fort” and “Agra Fort” will always be close, no matter how many prompts we write. To push further, we need the *vision* side to learn the subtle differences.

3.4.1 What We Fine-Tune (and What We Don’t)

Two of our 15 classes – Qutub Minar (94%) and Shalimar Bagh (100%) – are already handled perfectly by base CLIP. They have unique silhouettes (a tapering tower, a Mughal garden) that CLIP recognizes without help. Fine-tuning them would risk **catastrophic forgetting** for zero gain.

We define a **13-class specialist cluster** of confusable monuments and fine-tune only for those. The two easy classes remain on the zero-shot path.

3.4.2 Training Objective: Why Not Contrastive Loss?

Standard CLIP training uses InfoNCE (contrastive) loss, where images and texts in a batch form positive pairs, and all other combinations are negatives. In few-shot classification, this creates a problem: **two images of the same monument in the same batch become false negatives**. The loss pushes their embeddings apart – exactly the opposite of what we want.

Design Decision: Cross-Entropy over InfoNCE

We use **cross-entropy against frozen prompt-ensemble embeddings**. Each image is scored against one pre-computed text embedding per class. Same-class images are never penalized. This is a subtle but critical design choice for small datasets where class collisions within a batch are frequent.

3.4.3 Freeze Policy

Component	Status	Rationale
Vision blocks 0–8	Frozen	Preserve pre-trained low/mid-level features
Vision blocks 9–11	Trainable	Adapt high-level representations
Visual projection	Trainable	Align embedding space to Mughal features
Text tower + projection	Frozen	Keep text understanding general

Table 2: Freeze policy. Only 21.6M of 151.3M parameters are trainable.

3.4.4 Label Smoothing

We apply label smoothing ($\alpha = 0.1$) to the cross-entropy loss. With augmented data that is visually similar to its source images, the model can easily memorize training examples. Label smoothing prevents overconfidence and improves generalization to unseen viewpoints.

3.4.5 Hybrid Inference Pipeline

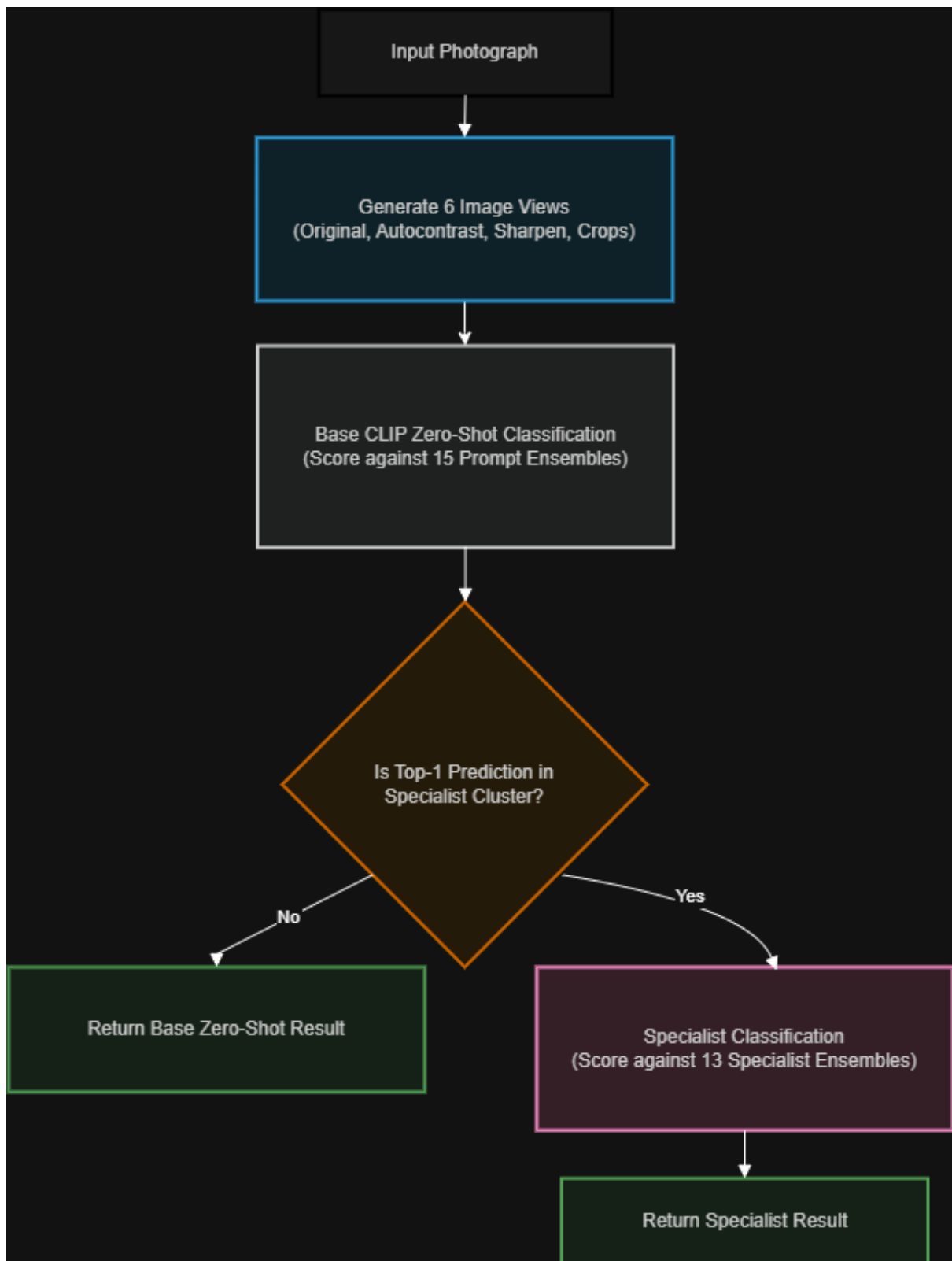


Figure 5: Inference pipeline. Base CLIP handles all 15 classes. If the prediction falls within the specialist cluster, the fine-tuned model re-evaluates with a higher confidence threshold (0.35 vs. 0.15).

3.5 The Nested Monument Problem

A unique challenge in this domain: some of our classes are *physically located inside* other classes. Buland Darwaza is the main gateway of Fatehpur Sikri. Moti Masjid is inside Agra Fort. A wide-angle photograph of Fatehpur Sikri will inevitably contain Buland Darwaza.

Is the model “wrong” when it classifies a Fatehpur Sikri photo as Buland Darwaza, if the gateway dominates the frame? We argue this is a **labelling ambiguity**, not a classification error. The ground truth is context-dependent.

Our Approach

We accept the overlap at the model level, and handle it in the UI. When the model predicts Buland Darwaza, the app displays a contextual note: *“Part of the Fatehpur Sikri complex.”* This gives the user correct information regardless of how the model resolves the ambiguity.

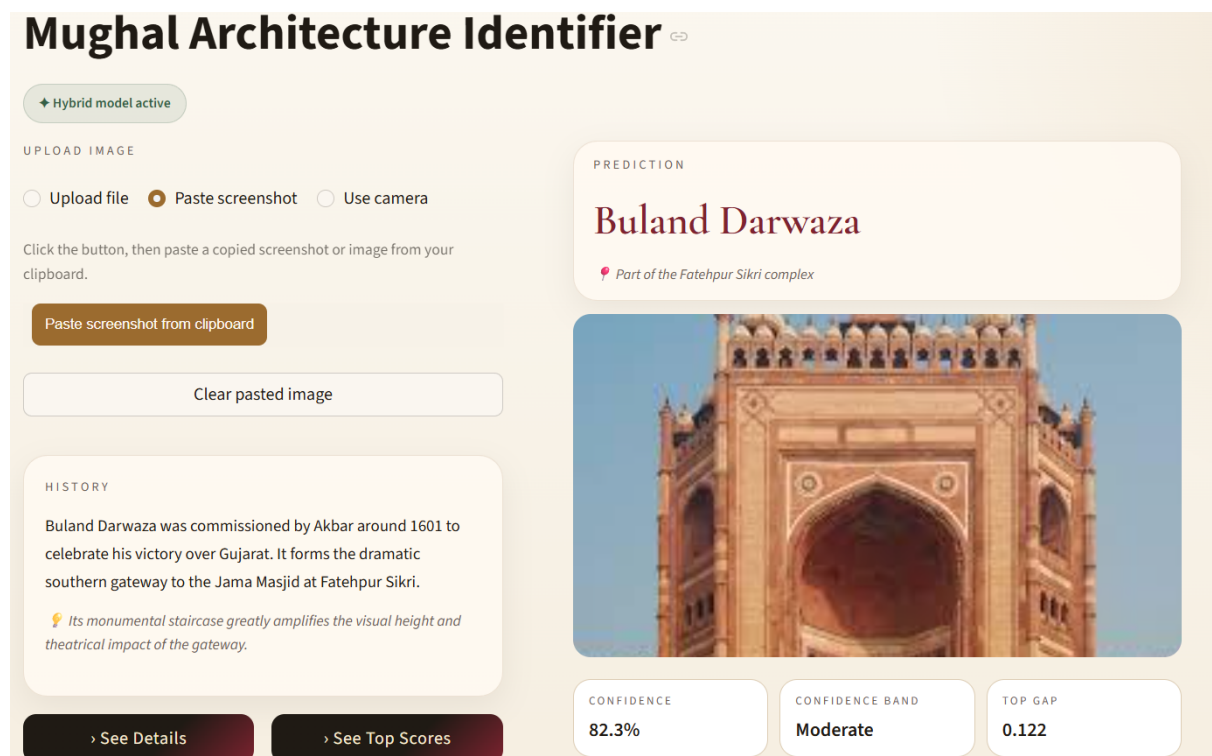


Figure 6: Example of the nested monument problem. Buland Darwaza appears within the larger Fatehpur Sikri complex, creating inherent ambiguity depending on framing and visual dominance, we ensure that we mention **Part of the Fatehpur Sikri complex** right below the prediction.

3.6 Deployment

The system is deployed as a Streamlit application with the full user flow: upload a photo → see the predicted monument → read its history → view visit information (hours, tickets) → open in Google Maps.

Key deployment decisions:

- **Dynamic OOD thresholds:** Base model uses 0.15 (random chance for 15 classes is ~6.7%); specialist uses 0.35 (fine-tuned models are more confident, so the bar must be higher).

- **Nested context notes:** Sub-monuments display their parent complex.
- **Model status badge:** The UI indicates whether the specialist model is active or the system is running on base CLIP only.

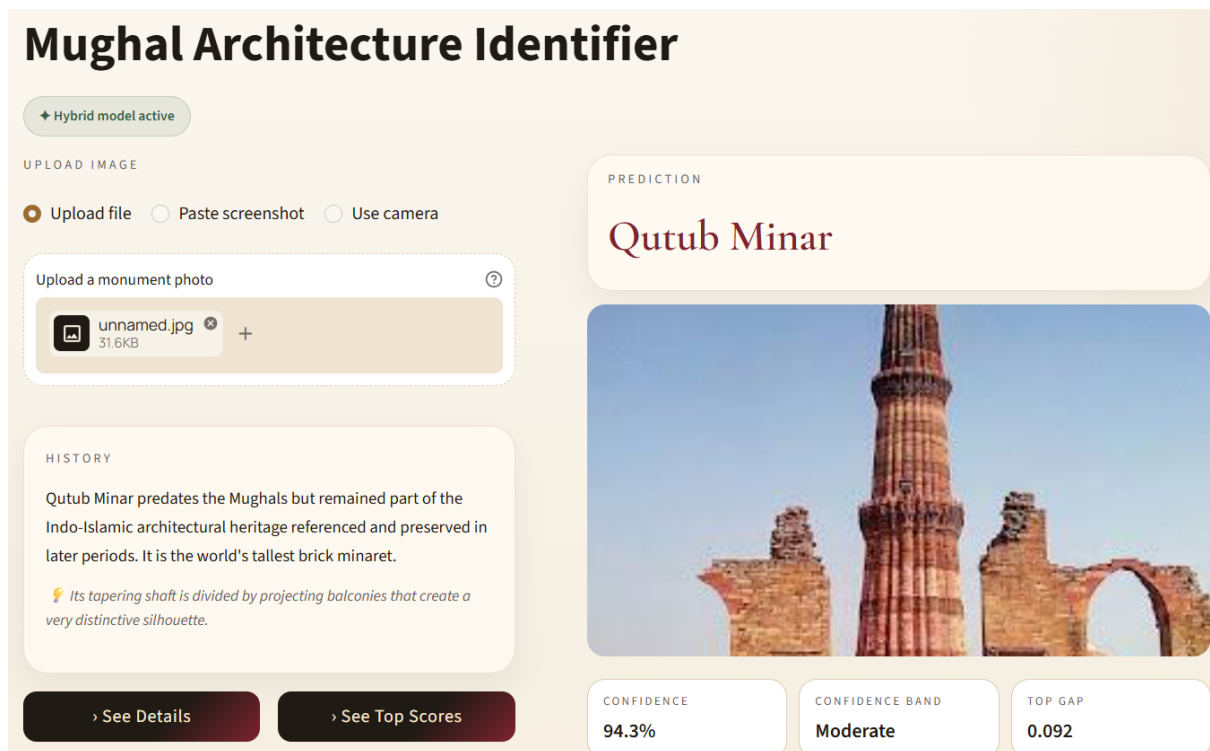


Figure 7: Main interface of the application showing a prediction for Qutub Minar along with the uploaded image, historical context, and confidence metrics (confidence score, confidence band, and top-gap).

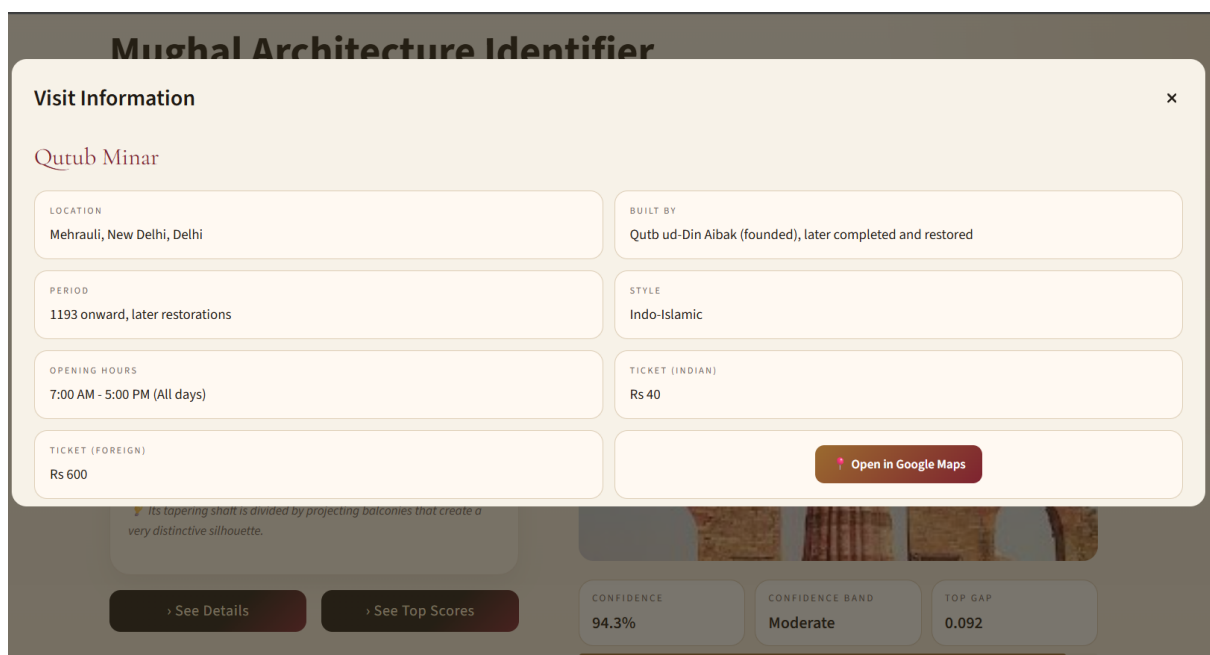


Figure 8: “See Details” view providing contextual visit information, including location, historical period, architectural style, ticket pricing, and a direct link to view the site on Google Maps.

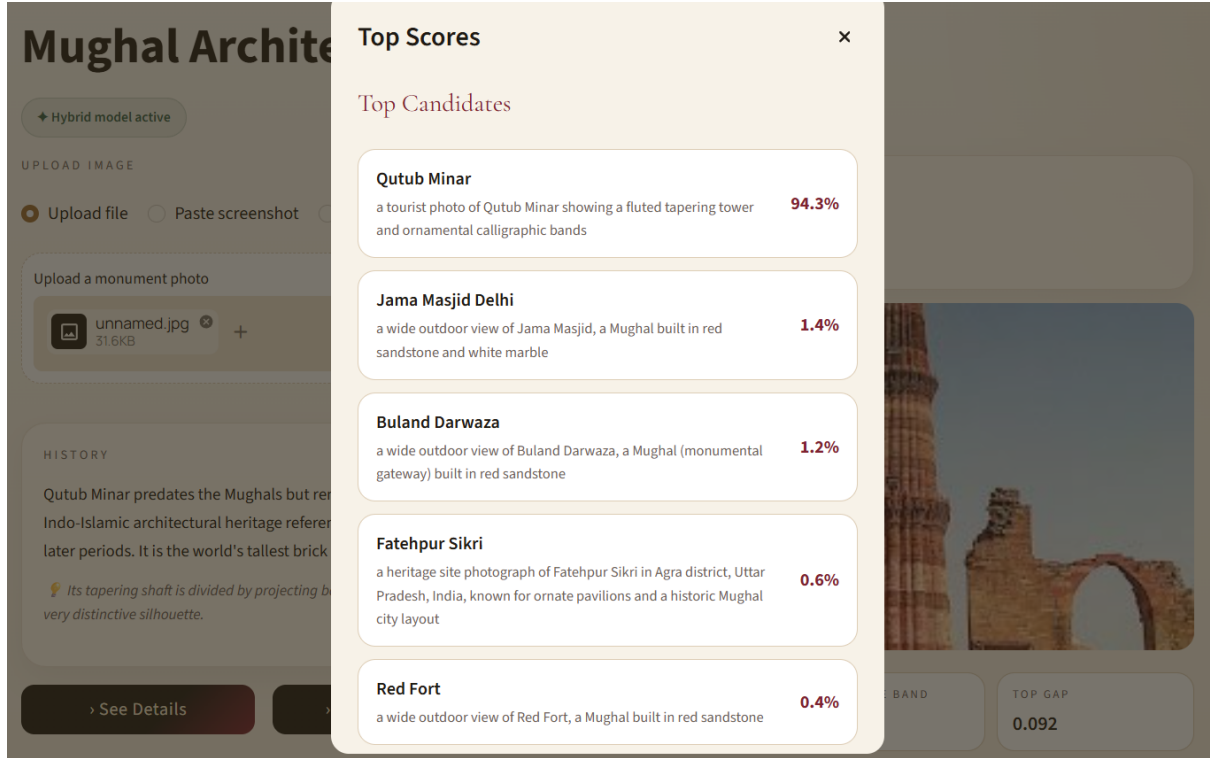


Figure 9: “See Top Scores” view displaying alternative model predictions with confidence scores, highlighting closely competing classes and providing insight into model uncertainty.

4 Results

Model	Top-1 Accuracy
Base CLIP (zero-shot, single prompt)	61.95%
Fine-tuned specialist (13 classes)	93.35%
K-fold mean test accuracy (7 folds)	88.38%
Best single fold (Fold 4, Epoch 8)	93.9%

Table 3: Aggregate results. The specialist model improves accuracy by 26.6 percentage points.

Class	Base CLIP	Fine-Tuned	Δ
Tomb of Salim Chishti	60.00%	100.00%	+40.00
Taj Mahal	91.98%	98.40%	+6.42
Bibi Ka Maqbara	53.75%	98.75%	+45.00
Humayun’s Tomb	79.52%	96.99%	+17.47
Safdarjung Tomb	57.50%	92.50%	+35.00
Jama Masjid Delhi	62.50%	92.50%	+30.00
Buland Darwaza	97.50%	93.75%	−3.75
Red Fort	30.00%	92.50%	+62.50
Akbar’s Tomb	2.50%	96.25%	+93.75
Itmad-ud-Daulah	37.50%	87.50%	+50.00
Moti Masjid Agra	35.00%	87.50%	+52.50
Fatehpur Sikri	48.75%	86.25%	+37.50
Agra Fort	20.00%	80.00%	+60.00

Why Buland Darwaza Regressed (and Why That's Good)

The base CLIP model was over-predicting Buland Darwaza — it used the "red sandstone gateway" concept as a catch-all for any red sandstone image. The fine-tuned model correctly learned to distinguish forts from gateways, which slightly reduced Buland Darwaza's recall but massively improved Agra Fort (20% → 80%) and Red Fort (30% → 92.5%). This is a net gain for the system.

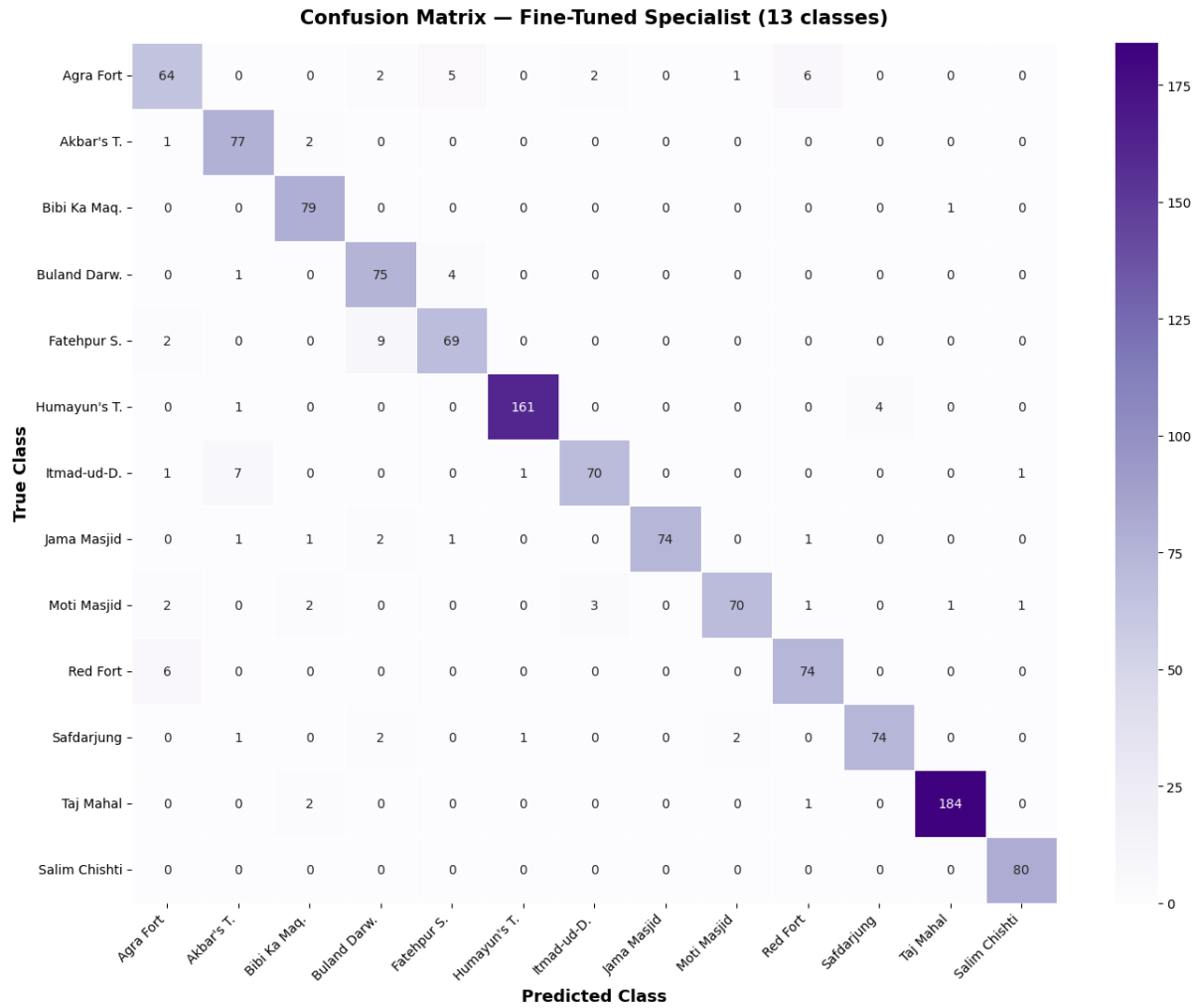


Figure 10: Confusion matrix after fine-tuning. The red sandstone cluster errors are dramatically reduced. Remaining confusions (Fatehpur Sikri ↔ Buland Darwaza) are architecturally co-located.

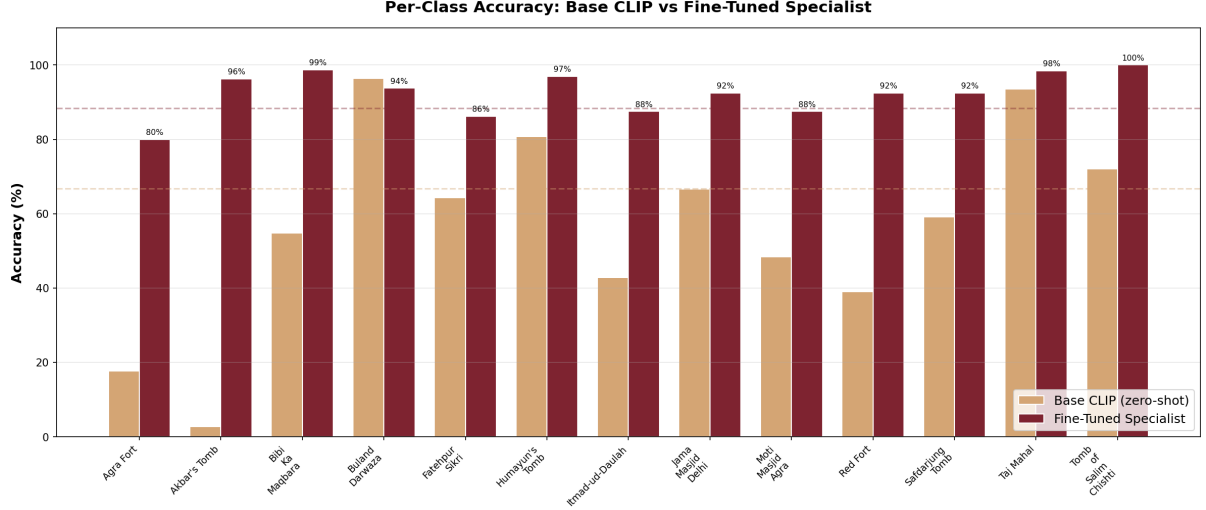


Figure 11: Per-class accuracy comparison. Every class except Buland Darwaza improves, with the largest gains in the previously confusable red sandstone cluster.

4.1 Where Our Model Still Fails

While the model performs well on most Mughal monuments, it struggles in cases where architectural styles overlap strongly across different sites. These failures are not random misclassifications but reflect a deeper limitation: the model relies purely on visual similarity without access to geographic or historical context.

A representative example is shown below, where an image of **Akbar's Tomb** is misclassified as **Jama Masjid, Delhi**. Both structures share Indo-Islamic architectural elements such as red sandstone construction and large arched façades, which can appear similar under certain viewpoints and lighting conditions. In this case, the model also reports relatively low confidence (62.9

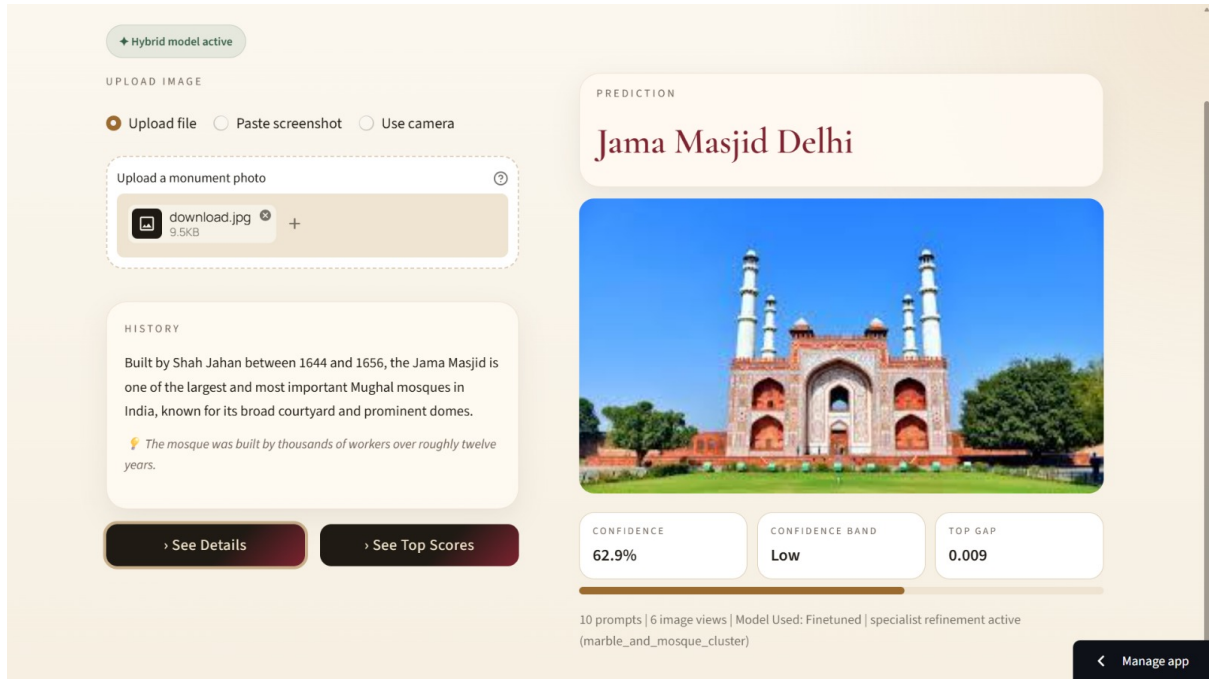


Figure 12: Failure case: the model predicts Akbar’s Tomb as Jama Masjid, Delhi with low confidence (62.9%). The visual similarity in red sandstone structure and arch-based façade leads to confusion under limited contextual information.

To better understand this limitation, we compare the same input with a modern multimodal large language model. Unlike our classifier, the LLM correctly identifies the monument as Akbar’s Tomb by leveraging broader contextual reasoning, including architectural cues and historical associations.

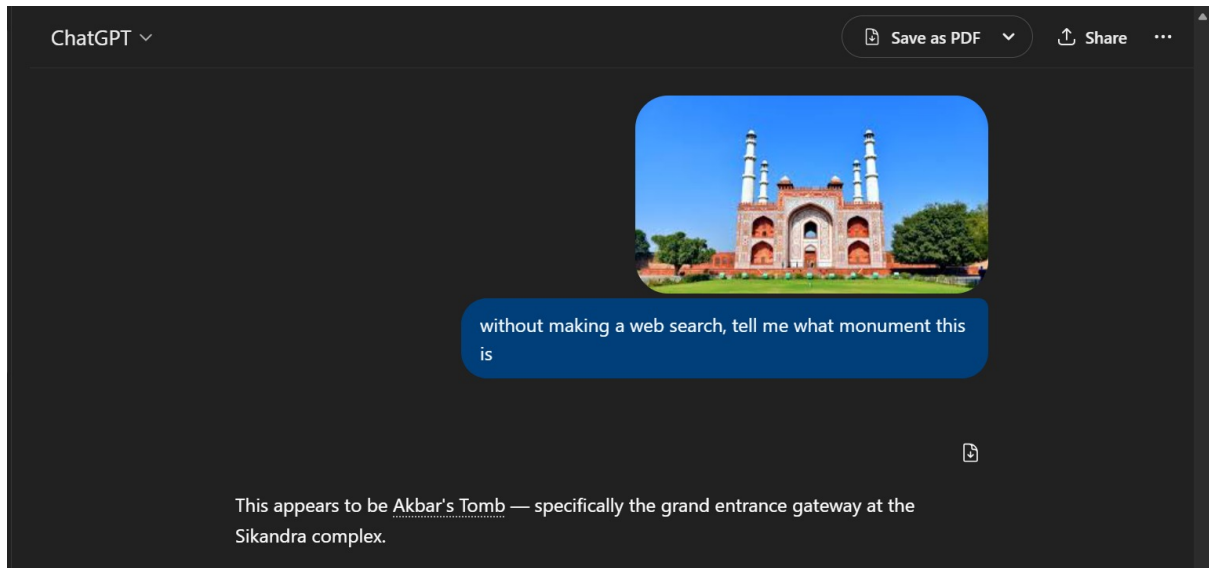


Figure 13: Comparison with a multimodal LLM, which correctly identifies the image as Akbar’s Tomb. The model uses contextual reasoning beyond visual similarity, incorporating historical and architectural knowledge.

Perception vs. Contextual Reasoning

Our model is a purely visual classifier and depends on learned feature similarity. In contrast, multimodal LLMs combine visual perception with external knowledge about geography and history. This allows them to resolve ambiguities where multiple monuments share near-identical visual characteristics, especially in Indo-Islamic architecture.

5 Limitations and Future Work

1. **More real data for weak classes.** Agra Fort (73.6%) would benefit most from additional unique photographs from diverse angles.
2. **Hierarchical classification.** Model Fatehpur Sikri as a *complex* containing sub-monuments, using a two-stage predict-then-refine approach.
3. **Hard negative mining.** Oversample training batches with known confusion pairs (Red Fort + Agra Fort) to force the model to learn their differences.
4. **Larger backbone.** ViT-L/14 would improve fine-grained discrimination at the cost of inference speed.
5. **Interior/night images.** Our dataset is dominated by daytime exterior shots. The model struggles with interior photographs and unusual lighting.

6 References

- [1] SMAI Lecture Slides, IIIT Hyderabad, Spring 2026
- [2] Blogs/articles/tutorials related to CLIP.
- [3] Acknowledgment: LLMs have been used to research, ideate and help create this submission.